NOVA PIONEER
SCHOOLS FOR INNOVATORS & LEADERS

Performance Task: Full-Stack Engineer, Curriculum Design Hub (PSSP)

Overview

We are excited to understand how you think as a full-stack engineer, how you turn an ambiguous product idea into a working and trustworthy system, and how you communicate technical decisions to engineers and educators alike.

Nova Pioneer is building an internal Curriculum Design Hub: an AI-powered tool that helps teachers generate, edit and publish lesson plans grounded in Nova Pioneer's curriculum structures, standards and ten years of existing learning materials. The teacher stays in the loop. AI drafts, the teacher shapes, and a curriculum lead approves before anything is shared with other teachers.

This task asks you to build a thin, working slice of that product.

Purpose of the Performance Task

This task is designed to help us understand:

- How you translate an ambiguous product idea into a working end-to-end system
- How you design a backend: data model, API boundaries and where logic lives
- How you integrate an LLM so its output is structured, grounded in source material and safe to fail
- How you handle authentication, roles and a publishing workflow
- How you use AI tools in your own engineering, and where you apply your own judgment
- How you communicate tradeoffs and decisions to a mixed audience

Format and Timing

- **Effort:** 5–6 hours. Please do not go beyond this. We would rather see a smaller slice done well, with clear notes on what you left out.
- **Submission:** GitHub (or similar) repo link plus Loom link
- **Files:** Codebase, README, AI_LOG.md, a `.env.example` and any configuration files required to run locally

Please submit your task by **Monday, October 26, 2026 at 09:00 am EAT**.

Task Context

Teachers at Nova Pioneer plan from shared schemes of work and assessments, but these stop short of lesson-level detail. Planning quality varies, and good lessons are rarely reused. The Hub should:

- Produce a strong first draft of a lesson plan quickly
- Keep that draft faithful to Nova Pioneer's curriculum
- Turn approved lessons into a reusable lesson bank

For this task, no real Nova Pioneer data is required. You will create a small mock curriculum corpus (see Part 2) and power the experience with it.

User Context

Assume two users:

- **Teacher:** a secondary school teacher in Kenya teaching under the Competency Based Curriculum (CBC). Planning time is short, often on a laptop between lessons, sometimes on unreliable connectivity. Wants a usable draft fast and full control to change it.
- **Curriculum Lead:** reviews submitted lesson plans for accuracy and alignment, then approves them or returns them with a comment.

Product Story

Build a Lesson Plan Generator

Design and implement a full-stack application in which a teacher picks a learning outcome, receives an AI-drafted lesson plan grounded in the curriculum corpus, edits it and submits it, and a Curriculum Lead approves it into a shared lesson bank.

1. Users, Roles and Access

- Two roles, **Teacher** and **Curriculum Lead**, with seeded demo accounts for each
- Authentication can be simple (email and password, magic link or a stubbed provider), but role checks must happen on the server, not only in the UI
- Teachers create and edit their own drafts and view published plans. They cannot see other teachers' drafts or publish.

2. Curriculum Corpus and Retrieval

Create a mock corpus of 8–12 short documents for one subject (for example, Grade 8 Integrated Science). Include at least two document types, such as scheme-of-work entries, learning outcomes and sample assessment items. Document the format.

- Ingest the corpus: chunk it, generate embeddings and store them (pgvector, SQLite with a vector extension, Chroma or an in-memory index are all fine)
- Retrieve the most relevant chunks for a request, with source references

3. Grounded Lesson Plan Generation

A teacher selects a learning outcome and lesson duration. The backend retrieves relevant curriculum content and asks an LLM to produce a lesson plan.

- Output must conform to a **lesson plan schema you define** (for example: objectives, materials, starter, main activities with timings, check for understanding, sources)
- Validate the output on the server and handle invalid output gracefully (retry, repair or a clear error)
- Every plan must cite the corpus chunks it drew on. A plan with no grounding should be flagged, not silently returned.

4. Edit, Submit and Publish

- Teachers edit the generated plan and save it. Previous versions are kept.
- Plans move through a defined workflow: Draft, Submitted, Returned, Published. Invalid transitions are rejected by the server.
- Published plans appear in a lesson bank that all teachers can view

5. Screens

A simple, functional UI covering sign-in, generating and editing a plan, a Curriculum Lead review queue and the lesson bank. This is a backend-heavy role, so screens should be clear, usable and accessible rather than polished. Aim for 2–4 key screens.

If you run short of time, prioritise Parts 2 and 3, then 1, then 4, then 5. Tell us in the README what you cut and why.

Technical Requirements

- Backend in TypeScript (Node) or Python; frontend in React with TypeScript (Next.js is fine for both)
- A real database (PostgreSQL preferred, SQLite acceptable) with migrations or a schema file
- Any LLM provider, plus a **mock mode** (deterministic, no API key) so we can run the app without credentials or cost
- The project should run locally with minimal setup (for example `docker compose up` or a short script), with seeded users and corpus
- Automated tests covering at least role-based access checks and LLM output validation
- Secrets in environment variables only, with a `.env.example`

Optional

- Streaming generation, token and cost logging, caching of repeat requests or a lesson-bank check before generating. These should be lightweight and well reasoned, not added at the expense of the core flow.

Engineering Expectations

This is a backend-heavy task: we care most about whether the system behaves correctly and safely. Please prioritise:

- Correctness of access control and workflow rules
- Lesson plans that stay grounded in the source material
- Clear failure handling when the LLM is slow, unavailable or wrong
- Readable code and a sensible structure

Consideration of data privacy (what is stored, and what is sent to the LLM provider) is a plus.

Use of AI Tools

You are encouraged to use AI coding assistants (Claude, Copilot, Cursor, ChatGPT or others). This is how we work, and we want to see how you work with them.

Because AI can generate a lot of plausible code quickly, we will not judge you on how much you build. We will judge you on whether you understand and can defend every part of what you submit, and on the decisions AI cannot make for you: what to build, what to leave out and how to know the system is working. We may ask you to walk through any part of your code.

Include a short **AI_LOG.md** (one page is enough) covering:

- Which tools you used and for which parts of the work
- At least one place where AI output was wrong, insecure or poorly suited, and how you caught and fixed it
- One decision you deliberately made yourself rather than accepting a suggestion, and why

Documentation & Loom Walkthrough

README

Include a concise README that explains:

- How to install and run the app locally, in mock mode and with a real LLM key
- Demo accounts and a suggested click-through
- The structure of the backend, data model and main components
- The corpus format and your lesson plan schema
- How a request becomes a grounded lesson plan, and what happens when generation fails
- Any assumptions, trade-offs, what you cut and extension ideas (e.g. scaling the corpus, versioning, cost control)

Loom (3–5 minutes)

Record a 3–5 minute Loom video that:

- Walks through the main flow as a Teacher, then as a Curriculum Lead
- Explains your key design and technical choices
- Shows what happens when generation fails or returns invalid output

Please include the Loom link in the README.

Deliverables

Your submission should include:

- A working full-stack application (backend, database and React + TypeScript frontend)
- A mock curriculum corpus and seed data
- A defined lesson plan schema, enforced on the server
- Automated tests for access control and LLM output validation
- A clear README and AI_LOG.md
- A 3–5 minute Loom walkthrough of your work

Nova Pioneer Culture Principles

JOY OF LEARNING. We are life-long learners, fueled by curiosity and discovery.

GREATER TOGETHER. We support our teammates and believe we can achieve more together.

ALWAYS GROWING. We seek difficult challenges, treat feedback as a gift, and see every failure as an opportunity to grow.

SERVANT LEADERSHIP. We put others first and engage our community with humility and generosity.

SOLUTIONS FIRST. We believe everything is possible when we are creative and think critically about the problem.

HIGH EXPECTATIONS. We sweat the small stuff, take pride in what we do, and set ambitious goals that we work hard to achieve.